NEURAL NETWORKS

Multilayer Perceptron:

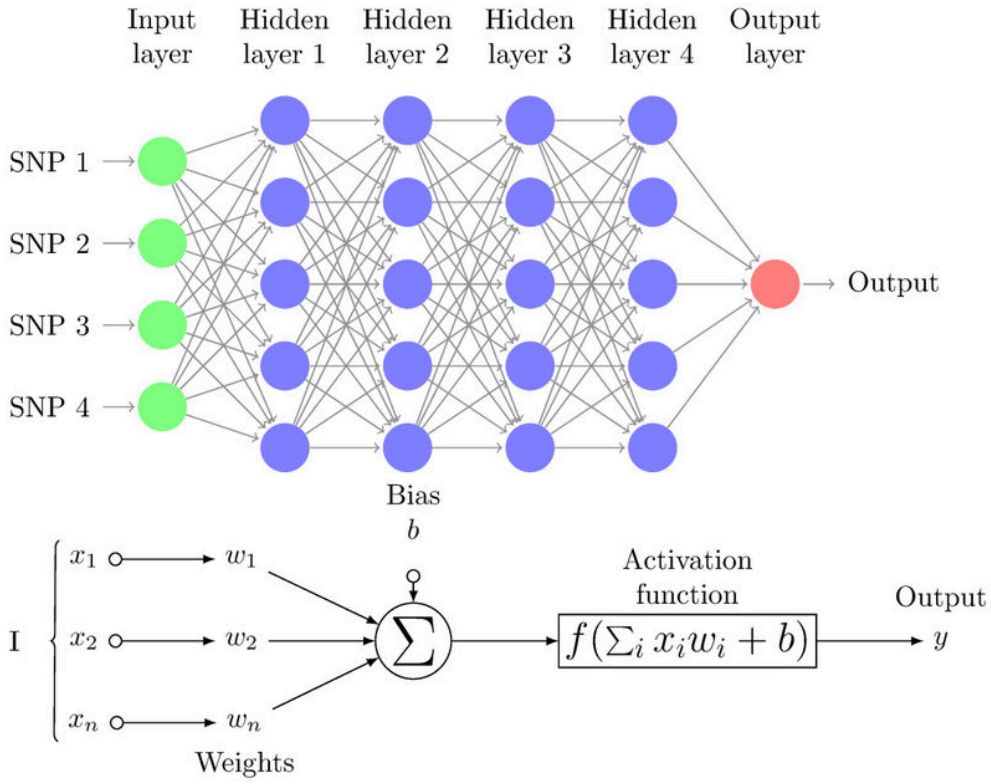
Basic Structure and Components

- A multilayer perceptron is a feed forward artificial neural network
- Consists of multiple layers of nodes (neurons) connected in a directed graph
- Each node, except input nodes, contains a nonlinear activation function
- Key components:
  - Neurons (computational units)
  - Weights (connection strengths between neurons)
  - Biases (offset values for each neuron)
  - Activation functions (introduce non-linearity)

Input, Hidden, and Output Layers

- **Input Layer**: Receives external data
  - One neuron per input feature
  - No computation performed, just passes data to the next layer
  - No activation function applied
- **Hidden Layers**: Intermediate processing layers
  - Perform computations on weighted inputs plus bias
  - Apply activation functions to introduce non-linearity
  - Extract and learn hierarchical features from data
  - Multiple hidden layers make the network "deep"
- **Output Layer**: Produces the final result
  - Structure depends on the problem type
  - For regression: Often one neuron per output variable

- For classification: Often one neuron per class (with softmax for multi-class)



Forward Propagation Process

1. **Initialize**: Input layer receives feature vector x
2. **Layer-by-Layer Computation**
3. **Output**: The final layer produces the prediction

Fully Connected Architecture

- Also known as "dense" layers
- Each neuron in a layer is connected to every neuron in the previous layer
- If a layer has n neurons and the previous layer has m neurons, there are n×m connections
- Each connection has an associated weight parameter

- Fully connected layers have the most parameters and are computationally intensive
- Enables the network to learn complex patterns across all inputs

Activation Functions:

Think of a **light switch**! 💡

- If the switch is OFF (0), no light.
- If the switch is ON (1), the light shines.
- But what if we want **dimmer control** instead of just ON/OFF?

👉 That's exactly what **activation functions** do! They decide whether a neuron should **activate** and by **how much**.

Types of Activation Functions

1️⃣ Linear Activation (Not Very Useful)

🟢 **Formula:** f(x)=x

- It simply returns the input as output.
- **Problem?** It can't capture **complex patterns** because deep layers behave the same as shallow layers.

📌 **Used for: Regression tasks (when output is a continuous number).**

2️⃣ Sigmoid (For Probability-Based Outputs)

🟢 **Formula:**

$$f(x) = \frac{1}{1 + e^{-x}}$$

- **Squashes values between 0 and 1** (great for probabilities).
- **Problem?** If inputs are too large or too small, the function **saturates** (gradients become near zero = slow learning).

📌 **Used in: Binary classification problems (Yes/No, 0/1 tasks).**

3️⃣ ReLU (Rectified Linear Unit – Most Popular!)

🟢 **Formula:**

$$f(x)=\max(0,x)$$

- If **x is positive, keep it**. If **x is negative, set it to 0**.
- **Solves vanishing gradient problem** for positive values.
- **Problem?** If x is always negative, neurons can die (**"Dead Neurons" issue**).

📌 **Used in: Most deep learning models (faster and better than Sigmoid/Tanh).**

4️⃣Softmax (For Multi-Class Classification 🎯)

🟢 **Formula:**

$$f(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

- Converts raw scores into **probabilities** (all values sum to 1).
- Helps decide which **category** an input belongs to.

📌 **Used in: Multi-class classification (e.g., detecting cat vs. dog vs. bird).**

🚀 Summary Table

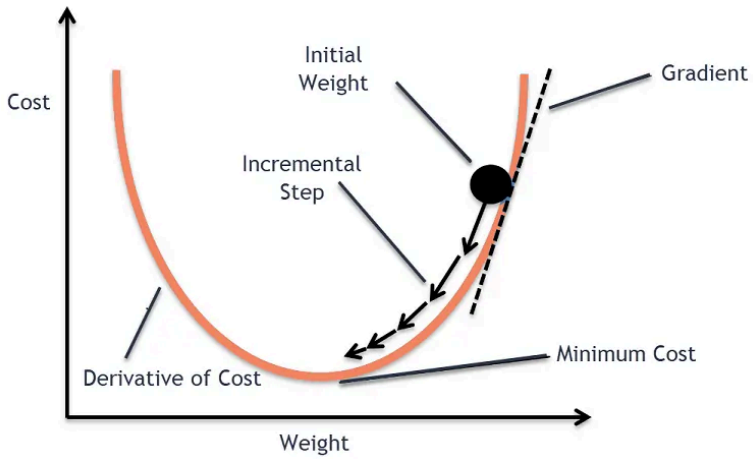
Activation Function	Output Range	Use Case
Linear	−∞-\infty to ∞\infty	Regression
Sigmoid	0 to 1	Binary classification
ReLU	0 to ∞\infty	Deep learning (CNNs, RNNs, etc.)
Softmax	0 to 1 (sums to 1)	Multi-class classification

Network Training Fundamentals:

- **Objective**: Find the optimal parameters (weights and biases) that minimize a loss function
- **Loss Function**: Measures the difference between predictions and actual targets
  - Common examples: Mean Squared Error (MSE), Cross-Entropy Loss.
- **Training Process**:
  1. Forward propagation to get predictions
  2. Calculate loss using a loss function
  3. Compute gradients through backpropagation
  4. Update parameters using an optimization algorithm
  5. Repeat until convergence or stopping criteria

Gradient Descent Optimization

- **Concept**: Iteratively adjust parameters in the direction of steepest descent of the loss function



- **Formula**: $\theta = \theta - \eta \nabla J(\theta)$
  - θ: Parameters (weights and biases)
  - η: Learning rate (step size)
  - $\nabla J(\theta)$: Gradient of the loss function with respect to parameters

- **Learning Rate**:
  - Controls the size of parameter updates
  - Too large: May cause divergence or oscillation
  - Too small: Slow convergence, may get stuck in local minima
  - Learning rate scheduling: Decreasing η over time can improve convergence

Stochastic Gradient Descent (SGD)

Imagine you're hiking down a mountain in the dark and want to reach the lowest point (the valley). You can't see the whole path, so after each step, you check the slope and decide the next move. This is exactly how **Stochastic Gradient Descent (SGD)** works in machine learning!

Gradient Descent?

Gradient Descent is an optimization algorithm used to find the **minimum** of a function (like the loss function in machine learning). It updates model parameters (weights) in the direction that reduces the error.

What Makes SGD Different?

There are three main types of Gradient Descent:

Type	How It Works	Pros	Cons
Batch Gradient Descent	Uses all data points to compute the gradient before updating weights.	More accurate updates	Slow for large datasets
Stochastic Gradient Descent (SGD)	Updates weights after each **single** data point.	Faster updates, works well for large datasets	Noisy updates
Mini-Batch Gradient Descent	Updates weights after a **small batch** of data points.	Balance between speed & accuracy	Requires tuning batch size

SGD is faster because it updates after every single data point instead of waiting for the whole dataset.

How SGD Works (Step-by-Step)

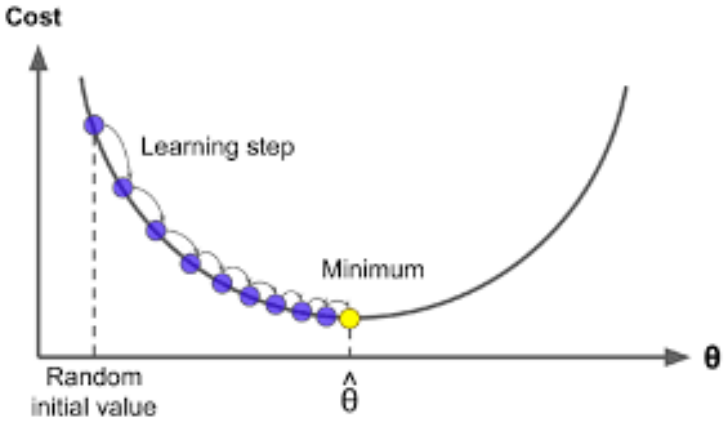
1️⃣ **Pick a random data point** from the dataset.

2️⃣ **Compute the gradient** (slope) of the loss function using that data point.

3️⃣ **Update the weights** in the opposite direction of the gradient.

4️⃣ **Repeat** for the next data point until the model converges.

💡 **SGD adds randomness**, which helps escape local minima and reach a better solution!



Example: SGD vs. Batch Gradient Descent

Imagine optimizing a loss function:

- **Batch Gradient Descent**: Moves in a smooth, direct path to the minimum.
- **SGD**: Takes a zigzag path but reaches the minimum much faster.

Advantages & Disadvantages of SGD

✅ **Pros:**

- Works well for **large datasets**.
- **Faster** than batch gradient descent.
- Helps escape **local minima** due to randomness.

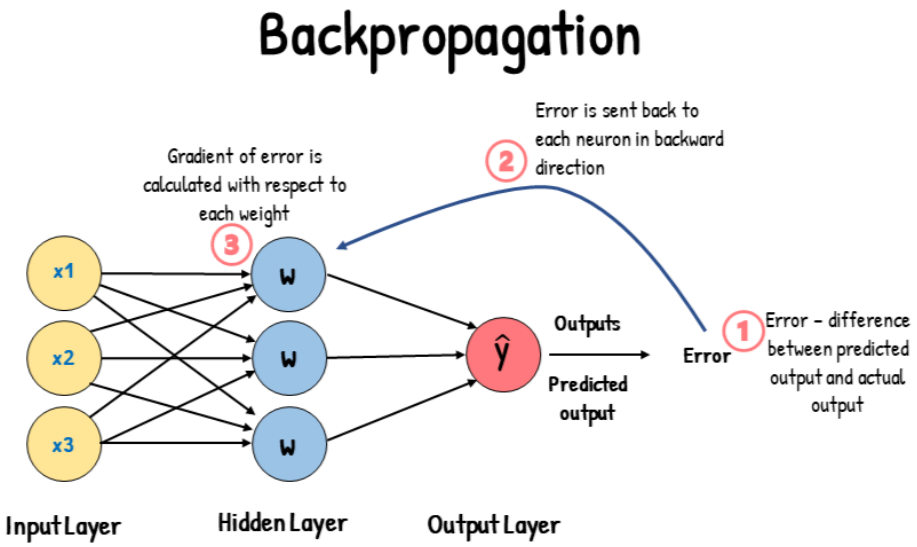
❌ **Cons:**

- Noisy updates can lead to instability.

- May **not converge smoothly** like batch gradient descent.
- Requires tuning the **learning rate** properly.

Error Backpropagation in Neural Networks:

Backpropagation is a supervised learning algorithm used for training neural networks. It efficiently updates the weights by propagating errors from the output layer back to the input layer using **Gradient Descent**.



Key Concepts

1.1 Forward Propagation

- Inputs are passed through the network, layer by layer.
- Activation functions transform weighted sums at each neuron.
- The final layer produces an **output**.
- The **error (loss)** is calculated by comparing the predicted output with the actual target.

1.2 Loss Function

A loss function measures the difference between predicted and actual values.

Common loss functions:

- **Mean Squared Error (MSE)** (for regression)

$$L = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- **Cross-Entropy Loss** (for classification)

$$L = -\sum y \log(\hat{y})$$

---

2. Backpropagation Algorithm Steps

Step 1: Compute Loss

The error (difference between predicted and actual value) is calculated.

Step 2: Compute Gradients (Backward Pass)

Using the **chain rule** of differentiation, we compute the gradient of the loss with respect to each weight in the network.

For a weight w, the gradient is:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$

where:

- $\frac{\partial L}{\partial \hat{y}}$ is the gradient of the loss w.r.t. the output.
- $\frac{\partial \hat{y}}{\partial w}$ is the gradient of the output w.r.t. the weight.

Step 3: Update Weights (Gradient Descent)

Using **Gradient Descent**, weights are updated:

$$w = w - \eta \cdot \frac{\partial L}{\partial w}$$

where η is the **learning rate**.

Step 4: Repeat Until Convergence

- Forward propagation → Loss computation → Backpropagation → Weight update.
- The process continues until the error is minimized.
- **Error signals travel backward**, adjusting weights layer by layer.

- **Earlier layers get smaller gradients** (this leads to the **vanishing gradient problem** in deep networks).
- Efficient optimization techniques like **Adam, RMSprop** help improve learning

---

🔷 **Forward propagation** computes predictions.

🔷 **Loss function** quantifies the error.

🔷 **Backpropagation** calculates gradients using the **chain rule**.

🔷 **Gradient Descent** updates weights to minimize loss.

Shallow vs. Deep Networks:

Think of a **shallow network** like a student who only studies from a **summary sheet**—they can learn basic concepts but may struggle with deeper understanding.

A **deep network**, on the other hand, is like a student reading an entire **textbook**, understanding details and complex patterns—but it takes more effort and time to train them.

Shallow Networks

- Have only **one or two hidden layers**.
- **Easy to train** (less computation, fewer parameters).
- Can **learn simple patterns** but struggle with **complex relationships**.

Example: A shallow network might recognize handwritten digits but struggle with detecting objects in an image.

Deep Networks

- Have **many hidden layers** (sometimes hundreds!).
- Can **learn complex patterns** (like human speech or facial recognition).
- **Harder to train** due to issues like:
    - **Vanishing gradients**: Early layers learn very slowly because updates become extremely small.

- **Exploding gradients**: If not controlled, updates become too large, making learning unstable.

Example: A deep network can learn **not just what an object looks like, but its shape, texture, and context**—like recognizing a cat in different lighting and poses.

Key Challenge: Vanishing & Exploding Gradients

When training a deep network, **errors must be passed back** from the output layer to earlier layers (backpropagation).

- If gradients **vanish** (become too small), early layers **stop learning** properly.
- If gradients **explode** (become too large), learning becomes **unstable**.

Unit Saturation & Vanishing Gradient Problem

Imagine you're passing a **message through a long chain of people**.

- If each person whispers the message very softly, by the time it reaches the last person, **the message is almost gone** (too weak to understand).
- This is similar to what happens in deep neural networks!

What is the Vanishing Gradient Problem?

When training deep networks, we use **backpropagation** to update weights. But if the gradient (the learning signal) gets **too small**, earlier layers **stop learning properly** because their updates become tiny.

🔷 Activation functions like **Sigmoid** and **Tanh** squeeze values into small ranges (0 to 1 or -1 to 1).

🔷 When inputs are large, these activations **flatten out**, meaning their slopes (gradients) become **almost zero**.

🔷 As we go back through the layers, the gradient **shrinks** more and more → **early layers stop learning**.

Effects of Vanishing Gradient

🚫 **Slow or no learning** in the first few layers.

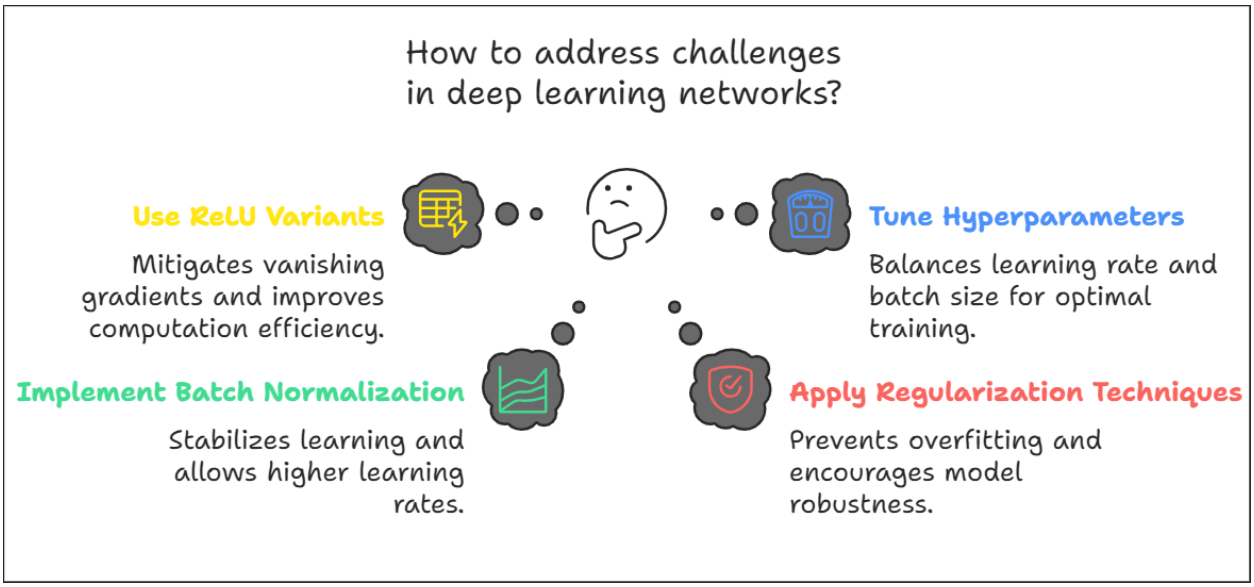
🚫 **Training deep networks becomes very difficult.**

Solution

✅ Use **ReLU activation** (it doesn't squash values, preventing vanishing gradients).

✅ Apply **Batch Normalization** (keeps values in a stable range).

✅ Use **better weight initialization techniques** to avoid very small values at the start.



ReLU (Rectified Linear Unit):

Imagine you are a teacher grading students.

- If a student scores **above 0 marks**, you **keep the score** as it is.
- If a student scores **below 0**, you just **give them a 0** instead of negative marks.

That's exactly how **ReLU** works!

ReLU Function: How It Works?

$$f(x) = \max(0, x)$$

- If **x is positive**, ReLU keeps it the same.
- If **x is negative**, ReLU turns it into **0**.

🔷 **Example:**

- f(5) = 5 ✅ (Positive values remain the same)
- f(-3) = 0 ❌ (Negative values become 0)

Why is ReLU Better?

✅ **Solves the vanishing gradient problem** for positive values.

✅ **Faster computation** (no complicated exponentials like Sigmoid or Tanh).

✅ Helps **train deep networks effectively**.

ReLU's Drawbacks & Variants

❌ **Problem:** If a neuron gets a negative value, it **always outputs 0** → **"Dead Neurons"** (they stop learning).

💡 **Solution:** Use **variants** of ReLU:

1️⃣ **Leaky ReLU** – Instead of 0, it gives **a small negative value** for negative inputs.

- Example: If , it outputs **0.03 instead of 0**.

  $x = -3$

2️⃣ **Parametric ReLU (PReLU)** – Learns the best small negative value automatically.

Summary

🔷 **ReLU helps deep networks learn better by preventing vanishing gradients.**

🔷 **Variants like Leaky ReLU, PReLU, and Swish solve its drawbacks.**

Hyperparameter Tuning for Deep Networks:

Think of training a deep network like **learning to ride a bicycle**. 🚴

- If you **pedal too fast**, you might lose control and fall.
- If you **pedal too slow**, you won't move forward much.

- You also need to **choose the right road** (smooth or bumpy) for better learning.

These choices are like **hyperparameters** in deep learning!

1️⃣ Learning Rate (How Fast We Learn)

- The **learning rate** decides **how big the updates** are when adjusting the weights.
- **Too high** → The model jumps around and never learns properly (**diverges** 🚀).
- **Too low** → The model learns very **slowly** and takes forever to reach good results (**long training time** 🦕).

✅ **Solution:** Find the right balance using **learning rate tuning**.

2️⃣ Batch Size (How Many Examples We Use at Once)

Think of **batch size** like ordering food:

- **Small batch** 🍕 (one slice at a time): More noise, but the model learns better generalization.
- **Large batch** 🍕🍕🍕 (whole pizza at once): Learning is more stable but may not generalize well.

🔷 **Small batches (e.g., 32, 64 samples)** → More noise, but better generalization.

🔷 **Large batches (e.g., 512, 1024 samples)** → More stable, but might overfit.

✅ **Solution:** Use a **moderate batch size** like 128 or 256 in most cases.

3️⃣ Optimizer Choice (How We Adjust the Weights)

Think of optimizers like **choosing the best road for a bicycle ride**.

🔷 **SGD (Stochastic Gradient Descent)** – Basic optimizer, like a **bumpy road**, can be slow but effective.

🔷 **Adam** – Adjusts the learning rate automatically, like a **smart bicycle that adjusts speed based on the road**.

✅ **Best Choice?** Most deep learning models use **Adam** for its **adaptive learning** and fast convergence.

🔷 **Learning Rate** → Too fast = unstable, too slow = takes forever.

🔷 **Batch Size** → Small = better generalization, Large = stable but may overfit.

🔷 **Optimizer Choice** → Adam is the most commonly used because it adapts well.

Batch Normalization (BN)

Imagine you're baking cookies 🍪.

- If the ingredients (flour, sugar, butter) **aren't measured properly**, some cookies might be too sweet, others too dry.
- **Batch Normalization** is like **standardizing** the ingredients, so every batch of cookies turns out **consistent**!

What is Batch Normalization?

When training deep networks, the **values inside each layer** (activations) can change too much, making learning unstable.

🔷 **Batch Normalization (BN)** fixes this by **scaling and shifting** the values so they stay in a **stable range**.

Why is BN Important?

✅ **Prevents Internal Covariate Shift** – Keeps activations from changing too much across layers.

✅ **Enables Faster Learning** – Allows **higher learning rates** without making training unstable.

✅ **Reduces Overfitting** – Works like a regularizer (less dependency on Dropout).

How Does It Work?

1️⃣ **Takes a batch of data** (e.g., 32 images).

2️⃣ **Calculates mean & variance** for that batch.

3️⃣ **Scales and shifts** the values so they follow a standard range.

4️⃣ **Helps the network learn faster & more reliably**!

Summary

🔷 BN **stabilizes learning** and allows using **higher learning rates**.

🔷 It also helps **prevent overfitting**, like a built-in regularizer.

🔷 Used in almost all modern deep networks for **faster and better training**

Regularization Techniques to Prevent Overfitting

Imagine you're studying for an exam 📚.

- If you **memorize** every question, you might do well on the practice test but fail the real exam (overfitting).
- Instead, if you focus on **understanding** concepts, you'll do better in any test (generalization).

Regularization helps a deep learning model **generalize** instead of just memorizing!

1️⃣ L1 & L2 Regularization (Avoiding Large Weights)

Think of **L1 & L2 Regularization** like packing for a trip 🎒:

- **L1 Regularization**: Forces you to **carry only necessary items** → results in some weights becoming exactly **zero** (sparse model).
- **L2 Regularization**: Limits the **size of all items** so no single item (weight) dominates → prevents very large weights.

✅ **L1 Regularization** → Encourages **simpler models** (some weights become 0).

✅ **L2 Regularization** → Keeps **all weights small** (prevents dominance of certain features).

2️⃣ Dropout (Turning Off Neurons Randomly)

Imagine a **sports team** where **only a few players** can play at a time.

- If every player gets a chance to play during training, **all players improve** and the team becomes **more balanced**.

- This is what **Dropout** does—it **randomly turns off neurons** during training so the model **doesn't rely too much on any single neuron**.

✅ Prevents the model from becoming **too dependent on certain neurons**.

✅ Forces the model to **learn more general patterns** instead of memorizing.

🔷 **L1 & L2 Regularization** → Prevent large or unnecessary weights.

🔷 **Dropout** → Turns off random neurons to encourage robustness.

🔷 **Both methods help prevent overfitting** and improve real-world performance